

PERÍODO: 202650 SI abril2026-agosto 2026
NOMBRE: Nayerli Montenegro
CARRERA: Electrónica y Automatización

PARCIAL: Segundo
CURSO (NRC): 28561
FECHA: 05 /06/ 2025

Enunciado del ejercicio

Problema 3.2.2: Matriz simétrica. Una matriz cuadrada A de orden $n \times n$ es simétrica si coincide con su traspuesta A^T ; es decir, $A = A^T$. Para este examen, asuma que el tamaño máximo de las matrices es 4×4 .

Instrucciones generales

1. Realice primero el análisis del problema antes de programar. Identifique qué significa que una matriz sea simétrica y cómo se compara A con su traspuesta A^T .
2. En hoja de análisis, escriba sus dos nombres y apellidos. Incluya obligatoriamente: tabla de objetos, definición de entradas y salidas, pseudocódigo y explicación breve de la lógica.
3. Desarrolle la función traspuesta en C. Esta función debe recibir por referencia la matriz original A , la matriz B donde se almacenará la traspuesta y el orden n . Recuerde que la traspuesta se obtiene cambiando filas por columnas: $B[j][i] = A[i][j]$.
4. Desarrolle la función simétrica en C. Esta función debe usar la función traspuesta y debe devolver 1 si la matriz es simétrica y 0 si no lo es.
5. Cree un programa principal que inicialice o lea una matriz cuadrada de máximo 4×4 , llame a las funciones anteriores y muestre un mensaje claro indicando si la matriz es simétrica o no.
6. Una vez que la profesora revise el pseudocódigo, puede pasar a Code::Blocks. No avance al código final si la lógica del análisis todavía no está clara.
7. En este documento coloque la captura de pantalla de la solución ejecutada en Code::Blocks, junto con el código fuente completo.
8. La entrega debe evidenciar coherencia entre tabla de objetos, pseudocódigo, funciones en C, ejecución y resultado final.

Tabla de objetos sugerida

Objeto	Nombre	Valor	Tipo
Matriz cuadrada original de máximo 4×4			
Matriz traspuesta de máximo 4×4			
Orden de la matriz			
Contador de filas			
Contador de columnas			

Indicador de simetría			
Valor uno: matriz simétrica			
Valor cero: matriz no simétrica			

Rúbrica de evaluación: lógica de resolución (20 puntos)

Criterio	Excelente	Bueno	En proceso	Puntaje
Comprensión del problema	Explica con claridad $A = A^T$ y reconoce que se comparan componentes correspondientes.	Reconoce la idea general de simetría, con explicación parcial.	Confunde simetría con igualdad de filas, columnas u otra condición.	4 pts
Tabla de objetos y análisis	Identifica correctamente matrices, contadores, orden n, indicador sim y tipos de datos.	Incluye la mayoría de objetos, con omisiones menores.	La tabla está incompleta o no se relaciona con el algoritmo.	4 pts
Diseño de la función traspuesta	Plantea correctamente $B[j][i] = A[i][j]$ usando ciclos anidados.	La lógica general es correcta, con pequeños errores de índices.	No logra transformar filas en columnas o altera datos incorrectamente.	4 pts
Diseño de la función simétrica	Compara A con B y devuelve 1 o 0 de forma coherente.	Compara matrices, pero con errores menores de control o retorno.	No compara todas las posiciones o no devuelve un resultado claro.	4 pts
Validación y pensamiento crítico	Prueba con casos simétricos y no simétricos, interpreta resultados y justifica errores posibles.	Presenta pruebas básicas, con justificación parcial.	No valida o solo muestra ejecución sin análisis.	4 pts

Preguntas de pensamiento crítico

1. Si una matriz no es simétrica, ¿es necesario seguir comparando todas sus posiciones después de encontrar la primera diferencia? Justifique su respuesta desde la eficiencia del algoritmo.
2. ¿Por qué es útil construir primero la función traspuesta antes de crear la función simétrica? Explique cómo la modularidad ayuda a probar y corregir el programa.

```
matriz-simetrica.c A
// Matrices de tamaño máximo 4x4 según la guía
float A[4][4]; // Matriz cuadrada
float AT[4][4];

// Solicitar el orden de la matriz G
printf("Ingrese el orden de la matriz (maximo 4): ");
scanf("%d", &n);

// Validación del tamaño de la matriz
if (n < 1 || n > 4) {
    printf("El orden debe estar entre 1 y 4.\n");
    return 1; // Terminar el programa si el orden es incorrecto
}

// 1. Lectura de la matriz original A usando scanf
printf("\n--- INGRESO DE DATOS ---\n");
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        printf("Elemento A[%d][%d]: ", i, j);
        scanf("%f", &A[i][j]); // %f se usa para leer números reales (float)
    }
}

// 2. Proceso de Transposición (Asignar a AT la transpuesta de A)
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        AT[j][i] = A[i][j];
    }
}

// 3. Verificación de simetría
bool es_simetrica = true;
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        if (A[i][j] != AT[j][i]) {
            es_simetrica = false;
            break;
        }
    }
}

// Resultado
printf("Resultado: La matriz %s es simétrica.\n", es_simetrica ? "SI" : "NO");

// Fin del programa
return 0;
}
```

```
#include <stdio.h>
#include <stdbool.h> // Para poder usar el tipo de dato bool (lógico)
```

```
int main() {
    // Definición de variables según la tabla de objetos de la ESPE
    int n; // Orden de la matriz (Dimensión)
    int i, j; // Contador de filas (i) y columnas (j)
    bool es_simetrica; // Indicador de simetría (true/false)

    // Matrices de tamaño máximo 4x4 según la guía
    float A[4][4]; // Matriz cuadrada
    float AT[4][4]; // Matriz transpuesta

    // Solicitar el orden de la matriz G
    printf("Ingrese el orden de la matriz cuadrada (maximo 4): ");
    scanf("%d", &n);

    // Validación del tamaño de la matriz
    if (n < 1 || n > 4) {
        printf("El orden debe estar entre 1 y 4.\n");
        return 1; // Terminar el programa si el orden es incorrecto
    }

    // 1. Lectura de la matriz original A usando scanf
    printf("\n--- INGRESO DE DATOS ---\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            printf("Elemento A[%d][%d]: ", i, j);
            scanf("%f", &A[i][j]); // %f se usa para leer números reales (float)
        }
    }

    // 2. Proceso de Transposición (Asignar a AT la transpuesta de A)
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            AT[j][i] = A[i][j];
        }
    }

    // 3. Verificación de simetría
    bool es_simetrica = true;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            if (A[i][j] != AT[j][i]) {
                es_simetrica = false;
                break;
            }
        }
    }

    // Resultado
    printf("Resultado: La matriz %s es simétrica.\n", es_simetrica ? "SI" : "NO");

    // Fin del programa
    return 0;
}
```

```

    for (j = 0; j < n; j++) {
        AT[j][i] = A[i][j]; // Intercambio de filas por columnas
    }
}

// 3. Verificación de Simetría
es_simetrica = true; // Se asume verdadera por defecto

for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
// Si un solo par de elementos difiere, ya no es simétrica

```

Elaborado por: Ing. Jenny A. Ruiz R.
 Docente TC DCCO